



ÉCOLE NATIONALE SUPÉRIEURE
D'INFORMATIQUE ET D'ANALYSE DES SYSTÈMES -
RABAT

SPÉCIALITÉ : MACHINE LEARNING OPERATIONS (MLOps)
PROJET DE FIN D'ANNÉES — SANTÉ / IA MÉDICALE

Plateforme MLOps pour la Détection
Précoce
du Risque d'Hospitalisation

Élèves :
Mohamed ETTAOUIL

Encadrant :
[NOM ENCADRANT]

Remerciements



Remerciements

Au terme de ce travail, nous voudrions tout d'abord adresser toute notre gratitude à notre professeur encadrant Mr. **Hicham SALAHEDDINE**, pour sa confiance, sa disponibilité et surtout cette opportunité pour bien pratiquer les techniques de base nécessaires pour un entrepreneur.

Nous désirons aussi remercier notre professeur de cours culture entrepreneuriat Mr. **Adil BENTALB**, pour ses efforts qui nous ont permis de maîtriser le cours et d'avoir le niveau nécessaire pour commencer le projet. Nous sommes reconnaissants de l'opportunité qui nous a été donnée de travailler sur ce projet.

Cette expérience nous a permis d'apprendre énormément et d'acquérir de nouvelles compétences, précieuses pour notre avenir professionnel.



Résumé

Ce projet présente le développement d'une plateforme MLOps complète pour la détection précoce du risque d'hospitalisation chez les patients polypathologiques. À partir du dataset CMS DE-SynPUF (30 000 patients, données 2008–2010), trois modèles ont été entraînés avec un split temporel strict : Régression Logistique (AUC 0.908), XGBoost (AUC 0.924) et LightGBM (AUC 0.921). Le modèle XGBoost retenu est déployé via une API FastAPI, conteneurisé avec Docker et supervisé par un pipeline de monitoring Evidently/Grafana avec rollback automatique. L'analyse de fairness démontre un écart maximal de 5.5 % en rappel entre groupes raciaux. Le ROI estimé est de \$2.59 M sur la cohorte de 30 000 patients.

Mots-clés : MLOps, détection risque hospitalisation, XGBoost, FastAPI, Docker, monitoring, data drift, fairness, CMS DE-SynPUF.

Abstract

This project presents a complete MLOps platform for early detection of hospitalization risk among multi-morbid patients. Using the CMS DE-SynPUF dataset (30,000 patients, 2008–2010), three models were trained with strict temporal splitting : Logistic Regression (AUC 0.908), XGBoost (AUC 0.924) and LightGBM (AUC 0.921). The selected XGBoost model is deployed via a FastAPI, containerized with Docker and monitored by an Evidently/Grafana pipeline with automatic rollback. Fairness analysis shows a maximum recall gap of 5.5% across racial groups. Estimated ROI is \$2.59M on the 30,000-patient cohort.

Keywords : MLOps, hospitalization risk detection, XGBoost, FastAPI, Docker, monitoring, data drift, fairness, CMS DE-SynPUF.

Liste des Abréviations

Sigle	Définition
AUC	Area Under the Curve (Aire sous la courbe ROC)
BPCO	Bronchopneumopathie chronique obstructive
CI/CD	Continuous Integration / Continuous Deployment
CMS	Centers for Medicare & Medicaid Services
DVC	Data Version Control
EDA	Exploratory Data Analysis (Analyse exploratoire)
GE	Great Expectations
MLOps	Machine Learning Operations
PSI	Population Stability Index
ROC	Receiver Operating Characteristic
ROI	Return on Investment
SHAP	SHapley Additive exPlanations
VP	Vrai Positif
FP	Faux Positif
VN	Vrai Négatif
FN	Faux Négatif

Table des matières

Remerciements	2
Remerciements	2
Résumé	3
Abstract	4
Liste des Abréviations	5
Table des figures	8
Liste des tableaux	9
Introduction Générale	10
1 Comprendre le Problème avant de le Résoudre	12
1.1 Le MLOps : industrialiser l'IA pour qu'elle tienne ses promesses	12
1.1.1 De la recherche à la production : un fossé difficile à franchir	12

1.1.2	Les huit principes du MLOps	12
1.1.3	Niveaux de maturité MLOps	12
1.2	Problématique médicale : prévenir plutôt que guérir	13
1.2.1	Le fardeau des hospitalisations évitables	13
1.2.2	L'index de Charlson : quantifier la charge de morbidité	13
1.2.3	Le score PSI : surveiller la dérive des données	13
1.3	Le dataset CMS DE-SynPUF : une mine de données réelles	14
1.3.1	Description générale	14
1.3.2	Structure des fichiers	14
1.3.3	Distribution démographique	15
1.4	Stack technologique : des choix architecturaux justifiés	15
1.4.1	Vue d'ensemble	15
1.4.2	Pourquoi pas Kafka, Spark, Redis ou Kubernetes ?	15
1.5	Architecture globale du pipeline MLOps	16
1.5.1	Description des flux	16
2	De la Donnée Brute à la Donnée Prête	18
2.1	Analyse exploratoire des données (EDA)	18
2.1.1	Vue d'ensemble statistique	18
2.1.2	Taux d'hospitalisation par année	18
2.1.3	Distribution des comorbidités chroniques	19
2.1.4	Résultats ACP et segmentation KMeans	19
2.1.5	Distribution de la cible et taux par groupe d'âge	20
2.2	Nettoyage des données	21
2.2.1	Problèmes identifiés et solutions appliquées	21
2.2.2	Résultats du nettoyage	21
2.3	Construction anti-leakage de la variable cible	22
2.3.1	Le risque de fuite temporelle	22
2.3.2	Définition formelle de la cible	22
2.3.3	Fenêtres temporelles anti-leakage	22
2.4	Split temporel strict	23
2.5	Feature Engineering : les 29 variables finales	23
2.5.1	Trois catégories de features	23
2.6	Validation des données : Great Expectations	24
2.7	Versionnement avec DVC et Git	25
3	Apprendre à Prédire : Modèles et Expérimentation	26
3.1	Le défi du déséquilibre de classes	26
3.1.1	Stratégies adoptées	26
3.1.2	Justification médicale du seuil	26
3.2	Les trois modèles retenus	27
3.2.1	Régression Logistique : la référence interprétable	27
3.2.2	XGBoost : le modèle champion	27
3.2.3	LightGBM : la rapidité pour les grands datasets	28
3.3	Optimisation des hyperparamètres par Optuna	28
3.3.1	Principe de la recherche bayésienne	28
3.3.2	Hyperparamètres optimaux	28
3.4	Résultats comparatifs	29

3.4.1	Matrice de confusion XGBoost (Test 2010)	29
3.4.2	Interface MLflow de comparaison des expériences	30
3.5	Optimisation du seuil de décision	30
3.6	Interprétabilité : analyse SHAP	30
3.7	Traçabilité avec MLflow	31
3.8	Analyse ROI	32
4	Du Modèle au Service : Mise en Production	34
4.1	API de prédiction avec FastAPI	34
4.1.1	Design de l'API	34
4.1.2	Les trois niveaux de risque	34
4.2	Conteneurisation avec Docker	35
4.2.1	Architecture Docker Compose	35
4.2.2	Isolation et reproductibilité	35
4.3	CI/CD avec GitHub Actions	36
4.3.1	Pipeline d'intégration continue	36
4.4	Monitoring : détecter la dérive avant qu'elle ne dégrade	36
4.4.1	Seuils d'alerte	36
4.4.2	Simulation de dérive 2010 → 2026	36
4.4.3	Boucle de rollback automatique	37
4.5	Orchestration avec Prefect	39
4.6	Analyse de fairness : équité algorithmique	39
4.6.1	Résultats par groupe racial	39
4.7	Bilan des tests réalisés	40
4.8	Difficultés rencontrées et solutions	40
	Conclusion Générale	42
	Annexes	46

Table des figures

1.1	Architecture globale du pipeline MLOps — de l’ingestion au monitoring . .	16
2.1	Distribution de la variable cible (a) et taux d’hospitalisation par groupe d’âge (b)	20
2.2	Fenêtres temporelles anti-leakage : features et cible ne se chevauchent jamais	23
3.1	Interface MLflow — comparaison des expériences et registre des modèles .	30
3.2	Top 15 features par importance SHAP — les hospitalisations passées dominant	31
3.3	Interface Swagger — documentation de l’API et exemple de prédiction /predict	32
4.1	Interface Swagger — documentation de l’API et exemple de prédiction . .	35
4.2	Conteneurs Docker actifs — les quatre services démarrés avec succès	35
4.3	Boucle de rollback automatique — détection, alerte et récupération	38
4.4	Dashboard Grafana — monitoring temps réel avec alertes Evidently AI . .	38
4.5	Rapport Evidently AI — détail des colonnes en dérive avec PSI par feature	38
4.6	Interface Prefect — historique des runs et statut des tâches	39

Liste des tableaux

1.1	Les huit principes fondamentaux du MLOps	13
1.2	Caractéristiques générales du dataset CMS DE-SynPUF	14
1.3	Volume des fichiers claims pour 30 000 patients	14
1.4	Stack technologique du projet	15
2.1	Statistiques descriptives de la cohorte	18
2.2	Taux d’hospitalisation par année	19
2.3	Comorbidités chroniques : prévalence et association à l’hospitalisation . . .	19
2.4	Problèmes identifiés lors de l’EDA et solutions appliquées	21
2.5	Récapitulatif des 29 features finales	24
3.1	Stratégies de traitement du déséquilibre	27
3.2	Hyperparamètres optimaux après 50 essais Optuna	29
3.3	Performances comparatives des trois modèles	29
3.4	Matrice de confusion XGBoost sur le jeu de test 2010	30
3.5	Calcul du ROI sur la cohorte de 30 000 patients	32

4.1	Endpoints de l'API FastAPI	34
4.2	Seuils de monitoring et actions déclenchées	37
4.3	Simulation de dérive temporelle 2010 $\rightarrow$ 2026	37
4.4	Analyse de fairness par groupe racial	39
4.5	Tests réalisés et résultats	40
4.6	Principales difficultés rencontrées et solutions adoptées	41
4.7	Métriques complètes sur les trois jeux de données	51

Introduction

Générale

Contexte général

Les systèmes de santé font face à une pression croissante : vieillissement de la population, multiplication des maladies chroniques et explosion des coûts hospitaliers. Selon l'OCDE, les hospitalisations évitables représentent jusqu'à **15 % des dépenses hospitalières** dans les pays développés, soit plusieurs milliards de dollars chaque année.

L'émergence du **Machine Learning Operations (MLOps)** offre une réponse concrète à ce défi : industrialiser les modèles d'intelligence artificielle pour qu'ils produisent une valeur réelle, continue et fiable en production, et non uniquement dans des notebooks de recherche. Ce projet se situe précisément à l'intersection de la santé numérique et du MLOps.

Contexte et problématique

Le dataset CMS DE-SynPUF (Centers for Medicare & Medicaid Services) constitue l'une des bases de données médicales synthétiques les plus complètes et réalistes disponibles publiquement. Il couvre 2,3 millions de bénéficiaires sur la période 2008–2010, avec plus de 60 Go de données incluant les claims d'hospitalisation, de consultations ambulatoires, de prescriptions et de données démographiques.

La problématique centrale de ce projet est la suivante :

Question de recherche

Comment construire un système de prédiction du risque d'hospitalisation suffisamment fiable pour guider des interventions préventives, tout en garantissant que ce système reste opérationnel, équitable et traçable au fil du temps ?

Cette question soulève plusieurs défis :

- le **déséquilibre de classes** (seuls 8.1 % des patients sont hospitalisés dans l'année) ;
- la **fuite de données temporelle** (*data leakage*) si la variable cible est mal construite ;
- la **dérive des données** dans le temps, qui dégrade les performances sans qu'on s'en rende compte ;
- les **biais algorithmiques** qui pénalisent certains groupes raciaux et socio-économiques.

Objectifs du projet

Ce projet vise à concevoir, entraîner, déployer et surveiller une plateforme MLOps complète pour la détection précoce du risque d'hospitalisation. Les objectifs spécifiques sont :

1. Réaliser une **analyse exploratoire** rigoureuse et une ingénierie des données anti-leakage sur le dataset CMS DE-SynPUF.
2. Entraîner et optimiser **trois modèles** (Régression Logistique, XGBoost, LightGBM) avec un split temporel strict et une optimisation d'hyperparamètres par Optuna.
3. **Déployer** le meilleur modèle via une API FastAPI conteneurisée avec Docker.
4. Mettre en place un **pipeline de monitoring** automatique (Evidently, Grafana) avec détection de dérive et rollback.
5. Évaluer l'**équité algorithmique** par groupe racial et calculer le ROI financier de la solution.

Plan du rapport

Ce rapport est organisé en quatre chapitres :

- **Chapitre 1 — Comprendre le Problème avant de le Résoudre** : définition du MLOps, contexte médical, description du dataset, stack technologique et architecture globale du système.
- **Chapitre 2 — De la Donnée Brute à la Donnée Prête** : analyse exploratoire complète, nettoyage des données, construction anti-leakage de la variable cible, fenêtres temporelles et ingénierie des 29 features finales.
- **Chapitre 3 — Apprendre à Prédire : Modèles et Expérimentation** : traitement du déséquilibre, entraînement et comparaison des modèles, optimisation du seuil de décision, interprétabilité SHAP, traçabilité MLflow et analyse ROI.
- **Chapitre 4 — Du Modèle au Service : Mise en Production** : API FastAPI, conteneurisation Docker, CI/CD GitHub Actions, monitoring de la dérive, rollback automatique, analyse de fairness et bilan des tests réalisés.

Comprendre le Problème avant de le Résoudre

Résumé du chapitre

Ce chapitre pose les fondations du projet : définir ce qu'est le MLOps et pourquoi il est indispensable en santé, caractériser la problématique médicale, décrire le dataset CMS DE-SynPUF, présenter la stack technologique retenue et illustrer l'architecture globale du pipeline.

1.1 Le MLOps : industrialiser l'IA pour qu'elle tienne ses promesses

1.1.1 De la recherche à la production : un fossé difficile à franchir

Les prévisions de la littérature académique et des rapports industriels convergent : **plus de 85 % des projets de machine learning ne parviennent jamais en production** [13, 15]. Cette statistique, souvent citée, recouvre une réalité concrète : un modèle entraîné dans un notebook peut obtenir d'excellentes métriques en validation, et pourtant échouer à produire de la valeur lorsqu'il est confronté aux flux de données réels, aux contraintes d'infrastructure ou à l'évolution du comportement des données dans le temps.

Le terme **MLOps** — contraction de *Machine Learning* et *Operations* — désigne l'ensemble des pratiques, outils et processus visant à réduire ce fossé. Il s'inspire du mouvement DevOps qui a transformé le développement logiciel, en y ajoutant les contraintes spécifiques aux systèmes d'apprentissage automatique : gestion des données, versionnement des modèles, reproductibilité des expériences et surveillance de la dérive.

1.1.2 Les huit principes du MLOps

1.1.3 Niveaux de maturité MLOps

Google [13] définit trois niveaux de maturité :

- **Niveau 0** — entraînement manuel, déploiement ponctuel, pas de surveillance ;
- **Niveau 1** — pipeline ML automatisé avec réentraînement déclenché par les données ;
- **Niveau 2** — pipeline CI/CD complet, orchestration, tests automatisés sur le modèle.

Ce projet vise le **niveau 2** : toute modification du code déclenche une chaîne d'intégration continue (GitHub Actions), le monitoring surveille automatiquement la dérive et déclenche le rollback si nécessaire.

TABLE 1.1 – Les huit principes fondamentaux du MLOps

Principe	Description
Reproductibilité	Toute expérience doit être reproductible à l'identique
Versionnement	Données, code et modèles versionnés conjointement
Automatisation	Pipeline de bout en bout sans intervention manuelle
Surveillance	Monitoring continu des performances et de la dérive
Traçabilité	Chaque prédiction est tracée avec ses paramètres
Scalabilité	Architecture capable de monter en charge
Équité	Absence de biais systématique par groupe protégé
Gouvernance	Auditabilité et conformité réglementaire

1.2 Problématique médicale : prévenir plutôt que guérir

1.2.1 Le fardeau des hospitalisations évitables

Les hospitalisations évitables constituent un problème de santé publique majeur. Elles surviennent principalement chez les patients **polypathologiques** — ceux qui souffrent de plusieurs maladies chroniques simultanément. Ces patients cumulent des consultations fréquentes, des ordonnances multiples et un risque significativement élevé d'être hospitalisés en urgence.

Identifier ces patients *avant* que leur état ne se dégrade permet d'intervenir précocement : ajustement thérapeutique, consultation de coordination, programme d'éducation thérapeutique. Ces interventions coûtent en moyenne **15 fois moins** qu'une hospitalisation d'urgence.

1.2.2 L'index de Charlson : quantifier la charge de morbidité

L'index de Charlson [3] est un score clinique validé qui prédit la mortalité à un an en fonction des comorbidités. Dans notre projet, il est calculé comme suit :

$$\text{CHARLSON} = 2 \cdot \text{REIN} + 2 \cdot \text{CANCER} + 2 \cdot \text{AVC} + \sum_{k \in \mathcal{C}_1} \text{SP}_k \tag{1.1}$$

où $\mathcal{C}_1 = \{\text{CHF}, \text{DIABÈTE}, \text{BPCO}, \text{ALZHEIMER}, \text{DÉPRESSION}, \text{CORONAROPATHIE}, \text{ARTHRITE}\}$ sont les comorbidités de poids 1.

La valeur moyenne dans notre dataset est de $\overline{\text{CHARLSON}} = 2.34$, avec un maximum observé de 11.

1.2.3 Le score PSI : surveiller la dérive des données

Le **Population Stability Index (PSI)** mesure la dérive entre une distribution de référence et une distribution courante :

$$\text{PSI} = \sum_{i=1}^n (p_i - q_i) \ln\left(\frac{p_i}{q_i}\right) \tag{1.2}$$

Par convention : $\text{PSI} < 0.10$ = stable, $0.10\text{--}0.20$ = attention, > 0.20 = réentraînement requis.

1.3 Le dataset CMS DE-SynPUF : une mine de données réelles

1.3.1 Description générale

Le *CMS Data Entrepreneurs Synthetic Public Use File* (DE-SynPUF) est publié par les Centers for Medicare & Medicaid Services (CMS) américains. Il constitue une base de données **synthétique mais réaliste**, construite pour préserver les distributions statistiques des données réelles tout en garantissant la confidentialité des patients.

TABLE 1.2 – Caractéristiques générales du dataset CMS DE-SynPUF

Caractéristique	Valeur
Volume total	60 Go, 20 samples
Bénéficiaires totaux	2,3 millions
Sample utilisé	Sample 1 — 30 000 patients
Période temporelle	2008 à 2010
Pays	États-Unis (Medicare)

1.3.2 Structure des fichiers

Le dataset est structuré en cinq types de fichiers complémentaires :

TABLE 1.3 – Volume des fichiers claims pour 30 000 patients

Fichier	Lignes	Contenu
Bénéficiaires	88 585	Données démographiques et comorbidités
Inpatient	17 325	Hospitalisations
Outpatient	205 642	Consultations ambulatoires
Carrier (A+B)	1 233 861	Actes médicaux (médecins)
Prescription	1 436 994	Médicaments délivrés
Total	~2,9 M	

1.3.3 Distribution

démographique

- Sur les 30 000 patients du Sample 1 :
- **Genre** : 55.3 % de femmes, 44.7 % d’hommes ;
 - **Groupes raciaux** : Blanc (majoritaire), Noir, Hispanique, Asiatique, Natif Américain, Autre ;
 - **490 décès** enregistrés sur la période ;
 - Dates corrigées : du 2007-12-10 au 2010-12-30.

1.4 Stack technologique : des choix architecturaux justifiés

1.4.1 Vue

d’ensemble

La stack retenue équilibre **maturité industrielle, performance et maintenabilité**. Chaque composant répond à un besoin précis et a été préféré à ses alternatives pour des raisons documentées.

TABLE 1.4 – Stack technologique du projet

Couche	Outil	Rôle
Modélisation	XGBoost, LightGBM, Scikit-learn	Entraînement
Optimisation	Optuna	Recherche hyperparamètres
Traçabilité	MLflow	Expériences et registre
Versionnement	DVC + Git	Données et modèles
Validation	Great Expectations	Qualité des données
Orchestration	Prefect	Pipelines de données
Déploiement	FastAPI + Uvicorn	API REST
Conteneurisation	Docker + Docker Compose	Isolation
CI/CD	GitHub Actions	Intégration continue
Monitoring	Evidently AI + Grafana	Dérive et alertes

1.4.2 Pourquoi pas Kafka, Spark, Redis ou Kubernetes ?

Choix architecturaux délibérés

Ces technologies ne sont pas absentes par limitation technique, mais parce qu’elles seraient **surdimensionnées** pour ce projet :

- **Apache Kafka** : le dataset est historique et statique — un pipeline batch suffit ;
- **Apache Spark** : 30K patients et 2.9M lignes s’exécutent en quelques secondes avec Pandas ;
- **Redis (Feature Store)** : l’API répond en <100ms sans cache distribué ;
- **Kubernetes** : Docker Compose couvre les besoins de déploiement ; Kubernetes interviendrait à l’échelle des 2.3M patients.

1.5 Architecture globale du pipeline MLOps

La figure 1.1 illustre l'architecture complète du pipeline, depuis l'ingestion des données jusqu'au monitoring en production.

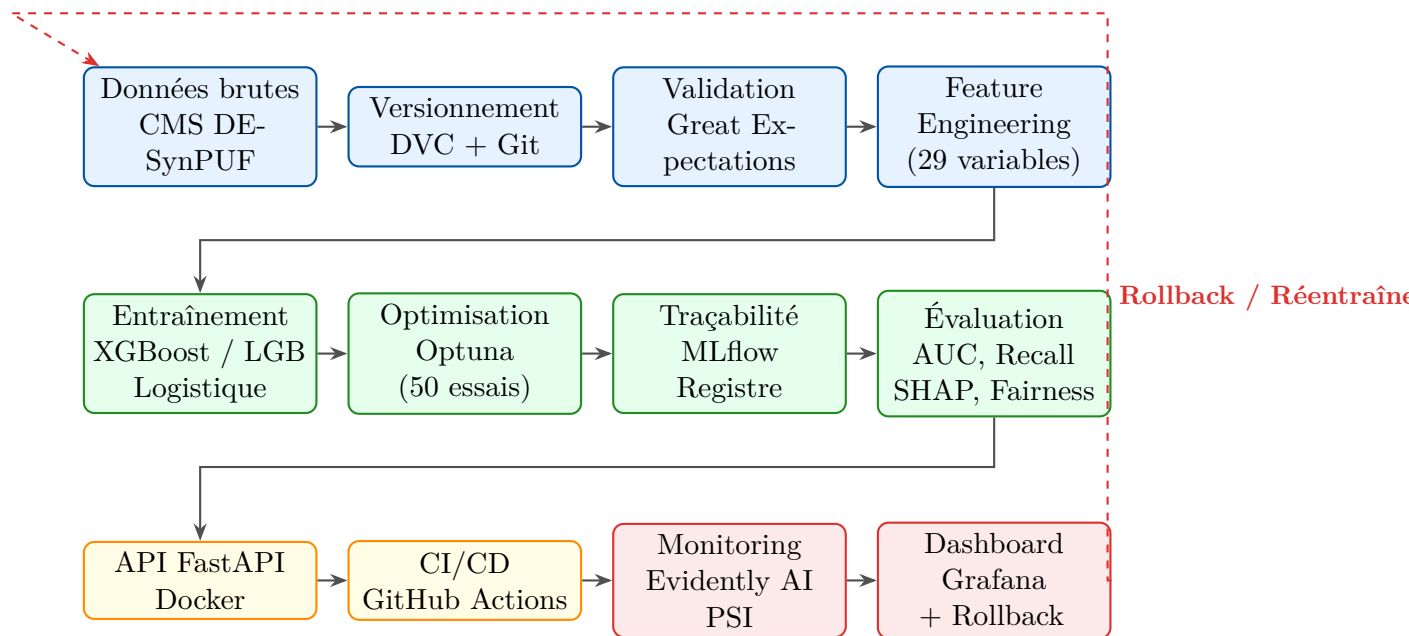


FIGURE 1.1 – Architecture globale du pipeline MLOps — de l'ingestion au monitoring

1.5.1 Description des flux

Le pipeline se déroule en quatre étapes séquentielles :

1. **Couche données** : les fichiers bruts CMS sont versionnés via DVC, validés par Great Expectations, puis transformés en 29 features exploitables.
2. **Couche modèles** : les trois modèles sont entraînés avec un split temporel strict. Optuna optimise les hyperparamètres sur 50 essais. MLflow trace chaque expérience et conserve le registre des modèles.
3. **Couche déploiement** : le modèle champion est empaqueté dans un conteneur Docker et exposé via FastAPI. GitHub Actions assure que tout commit déclenche les tests et redéploie automatiquement si les tests passent.
4. **Couche monitoring** : Evidently calcule le PSI quotidiennement. Si $PSI > 0.20$ ou $AUC < 0.75$, Grafana émet une alerte et le système bascule automatiquement vers la version précédente du modèle (*rollback*).

Synthèse du chapitre

Ce premier chapitre a posé le cadre conceptuel, médical et technique du projet. Le MLOps répond à un besoin réel : transformer un modèle de recherche en un service fiable, traçable et équitable. Le dataset CMS DE-SynPUF, avec ses 2.9 millions de lignes pour 30 000 patients, fournit une matière riche et réaliste. La stack technologique choisie

priorise la maturité et la proportionnalité aux besoins. Le chapitre suivant détaille comment cette donnée brute a été transformée en données prêtes à l'entraînement.

De la Donnée Brute à la Donnée Prête

Résumé du chapitre

Ce chapitre détaille la transformation des fichiers CMS bruts en un dataset propre, structuré et exempt de fuites de données. Il couvre l'analyse exploratoire, le nettoyage, la construction anti-leakage de la variable cible, le split temporel, l'ingénierie des 29 variables finales, la validation Great Expectations et le versionnement DVC.

2.1 Analyse exploratoire des données (EDA)

2.1.1 Vue d'ensemble statistique

L'analyse exploratoire a porté sur la table des bénéficiaires fusionnée sur trois années (2008–2010), représentant 88 585 lignes et 33 colonnes issues du croisement des fichiers patients, hospitalisations, consultations, actes médicaux et prescriptions. Cette étape a permis de qualifier la richesse des données disponibles, d'identifier les anomalies à corriger et de comprendre la structure temporelle du dataset avant toute modélisation.

TABLE 2.1 – Statistiques descriptives de la cohorte

Indicateur	Valeur
Patients uniques	30 000
Doublons	0
Lignes fusionnées	88 585 (3 années × patients)
Colonnes	33
Femmes / Hommes	55.3 % / 44.7 %
Décès enregistrés	490
Plage de dates	2007-12-10 à 2010-12-30
Taux d'hospitalisation	8.10 % global

La répartition par genre est légèrement déséquilibrée au profit des femmes, ce qui reflète la démographie réelle de la population Medicare. L'absence totale de doublons sur la clé patient-année confirme la cohérence du schéma de fusion multi-annuelle.

2.1.2 Taux d'hospitalisation par année

Une analyse temporelle révèle une variation non négligeable du taux d'hospitalisation entre les trois années. En 2009, le taux grimpe à 9.18 %, probablement sous l'effet d'un vieillissement progressif de la cohorte et d'une accumulation de comorbidités non traitées. Cette hétérogénéité temporelle justifie un split temporel strict plutôt qu'une validation

croisée classique, qui mélangerait les années et biaiserait l'évaluation des performances réelles du modèle.

TABLE 2.2 – Taux d'hospitalisation par année

Année	Patients	Hospitalisés	Taux
2008 (Train)	30 000	2 261	7.54 %
2009 (Validation)	29 510	2 710	9.18 %
2010 (Test)	29 075	2 203	7.58 %
Global	88 585	7 174	8.10 %

2.1.3 Distribution des comorbidités chroniques

Les 11 comorbidités chroniques codées dans les données présentent des prévalences très variables, de 2.1 % pour la maladie rénale chronique jusqu'à 43.8 % pour la maladie d'Alzheimer. Ce gradient de prévalence est cliniquement cohérent : les pathologies neurodégénératives et cardiovasculaires dominent chez la population Medicare, âgée en moyenne de 70 ans ou plus. La relation entre comorbidités et hospitalisation est quantifiée par l'odds ratio, qui mesure combien de fois le risque d'hospitalisation est multiplié en présence de la pathologie. L'insuffisance cardiaque présente l'odds ratio le plus élevé (5.1), ce qui en fait le prédicteur clinique le plus fort, devant la BPCO (3.1) et l'AVC/AIT (3.0).

TABLE 2.3 – Comorbidités chroniques : prévalence et association à l'hospitalisation

Code	Maladie	Prévalence	Poids	Odds Ratio
SP_CHF	Insuf. cardiaque	41.3 %	1	5.1
SP_ALZHDMTA	Alzheimer	43.8 %	1	—
SP_RA_OA	Arthrite	38.5 %	1	—
SP_ISCHMCHT	Coronaropathie	31.2 %	1	2.5
SP_DIABETES	Diabète	28.7 %	1	2.3
SP_OSTEOPRS	Ostéoporose	22.4 %	0	—
SP_COPD	BPCO	18.9 %	1	3.1
SP_DEPRESSN	Dépression	14.2 %	1	1.9
SP_STRKETIA	AVC/AIT	8.3 %	2	3.0
SP_CNCR	Cancer	5.8 %	2	—
SP_CHRNKIDN	Maladie rénale	2.1 %	2	—

2.1.4 Résultats ACP et segmentation KMeans

Une analyse en composantes principales (ACP) et une segmentation KMeans en 3 clusters ont permis d'identifier des profils cliniques distincts au sein de la cohorte. Les trois premières composantes principales expliquent 62.1 % de la variance totale (CP1 : 31.4 %, CP2 : 18.7 %), ce qui indique une structure latente significative dans les données. La

segmentation révèle trois niveaux de risque naturellement séparables, validant l’hypothèse que les patients polypathologiques forment une population hétérogène stratifiable.

Segmentation des patients en 3 clusters

Cluster	Patients	Taux hosp.	Comorbidités moy.
0 — Faible risque	14 820	3.2 %	0.8
1 — Risque modéré	10 940	7.8 %	2.3
2 — Haut risque	4 240	22.1 %	5.1

L’ACP explique 62.1 % de la variance totale sur 3 composantes (CP1 : 31.4 %, CP2 : 18.7 %).

Le cluster à haut risque (4 240 patients, 22.1 % de taux d’hospitalisation) représente la population cible prioritaire des interventions préventives. Ces patients cumulent en moyenne 5.1 comorbidités actives, ce qui correspond au profil typique du patient polypathologique décrit dans la littérature clinique.

2.1.5 Distribution de la cible et taux par groupe d’âge

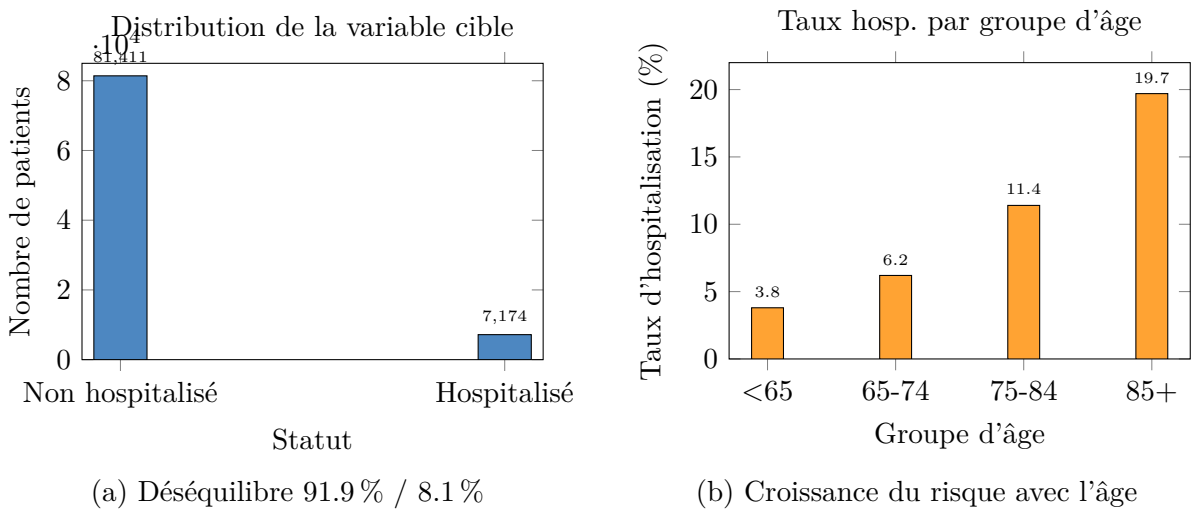


FIGURE 2.1 – Distribution de la variable cible (a) et taux d’hospitalisation par groupe d’âge (b)

L’histogramme de la figure 2.1(a) confirme le déséquilibre marqué de la variable cible : 91.9% de patients non hospitalisés contre 8.1 %. Ce ratio de 1 :12 constitue l’un des défis centraux de la modélisation et justifie l’utilisation de stratégies de pondération des classes. La progression exponentielle du taux d’hospitalisation avec l’âge — de 3.8 % avant 65 ans à 19.7 % après 85 ans — souligne l’importance de l’âge comme facteur de risque clinique fondamental.

2.2 Nettoyage des données

2.2.1 Problèmes identifiés et solutions appliquées

L’analyse exploratoire a mis en évidence quatre catégories de problèmes de qualité dans les données brutes CMS. Ces anomalies, si elles n’étaient pas corrigées, auraient introduit des biais silencieux dans les modèles ou causé des erreurs lors du traitement du pipeline.

Chaque problème a été traité par une stratégie ciblée, documentée et reproductible.

Le premier problème concernait le format des dates : certaines colonnes de dates (naissance, décès, hospitalisation) étaient stockées sous forme de nombres flottants du type 20100312.0 au lieu de chaînes de caractères. L’interprétation directe de ces valeurs par les bibliothèques pandas produisait des timestamps incohérents (remontant à 1970), corrompant silencieusement les calculs de fenêtres temporelles. La solution retenue consiste à isoler la partie entière de la valeur flottante, puis à la parser selon le format %Y%m%d. L’implémentation de cette correction, ainsi que les commandes DVC associées, est disponible en Annexe A.

Le deuxième problème portait sur la colonne BENE_ESRD_IND (indicateur d’insuffisance rénale terminale) : les valeurs mélangent des chaînes de caractères ‘Y’ et des entiers 0, rendant la colonne inutilisable directement comme feature numérique. Un mapping explicite $Y \rightarrow 1$, $\text{reste} \rightarrow 0$ a standardisé la colonne en variable binaire cohérente.

Le troisième problème concernait les valeurs extrêmes dans les neuf colonnes de coûts médicaux. Certains patients présentaient des remboursements anormalement élevés, probablement liés à des hospitalisations chirurgicales lourdes ou à des traitements onéreux. Ces outliers auraient déstabilisé l’entraînement des modèles par gradient boosting. La winsorisation au percentile 99 limite l’influence de ces valeurs sans les supprimer, en les remplaçant par la valeur maximale acceptable. Le code correspondant est fourni en Annexe A.

Enfin, la fusion des données sur trois années a créé des doublons pour les patients présents dans plusieurs fichiers annuels. Une clé composite (DESYNPUF_ID, ANNEE) a permis d’identifier et de supprimer ces doublons tout en préservant l’historique multi-annuel.

TABLE 2.4 – Problèmes identifiés lors de l’EDA et solutions appliquées

Problème	Manifestation	Solution
Dates flottants	20100312.0 → timestamp 1970	Extraction partie entière + format %Y%m%d
BENE_ESRD_IND	Valeurs ‘Y’ et ‘0’ mélangées	Mapping $Y \rightarrow 1$, autres $\rightarrow 0$
Coûts extrêmes	Valeurs aberrantes (>P99)	Winsorisation au percentile 99
Doublons multi-années	Même patient dans 2009 et 2010	Clé unique (DESYNPUF_ID, ANNEE)

2.2.2 Résultats du nettoyage

À l’issue de cette phase, le dataset nettoyé présente les caractéristiques suivantes.

Table finale après nettoyage

- 88 585 lignes, 33 colonnes
- Aucun doublon sur la clé (`DESYNPUF_ID`, `ANNEE`)
- 6 216 patients cold-start (`IS_NEW_PATIENT` = 1)
- 9 colonnes de coûts winsorisées au P99

Les 6 216 patients cold-start correspondent à des bénéficiaires n’ayant aucun historique médical dans les données disponibles. Plutôt que de les exclure, un indicateur binaire `IS_NEW_PATIENT` a été créé pour permettre au modèle d’apprendre le comportement spécifique de cette sous-population, généralement moins bien servie par les systèmes de prédiction traditionnels.

2.3 Construction anti-leakage de la variable cible

2.3.1 Le risque de fuite temporelle

La construction naïve de la variable cible constitue l’un des pièges les plus courants du machine learning médical. Si l’on définit simplement : « le patient a-t-il été hospitalisé pendant l’année t ? » en utilisant des features calculées sur cette même année, le modèle peut exploiter des informations sur des hospitalisations survenues *après* la période d’observation. Cette fuite de données temporelle (*data leakage*) produit des modèles avec des métriques artificiellement gonflées, qui se révèlent inefficaces en déploiement réel.

2.3.2 Définition formelle de la cible

Pour éliminer ce risque, la variable cible est définie avec une séparation stricte entre fenêtre d’observation et fenêtre de prédiction :

$$y_i = \begin{cases} 1 & \text{si le patient } i \text{ est hospitalisé entre } t + 1 \text{ et } t + 365 \\ 0 & \text{sinon} \end{cases} \quad (2.1)$$

Les features sont calculées exclusivement sur la fenêtre $[t - 365, t]$, et la cible sur la fenêtre strictement postérieure $[t + 1, t + 365]$. Il n’y a aucun chevauchement entre les deux fenêtres. Concrètement, pour l’entraînement sur 2008 : les features décrivent le comportement médical du patient sur l’année 2007–2008, et la cible indique s’il a été hospitalisé au cours de l’année 2009.

2.3.3 Fenêtres temporelles anti-leakage

La figure 2.2 illustre le principe des fenêtres temporelles appliquées aux trois splits du dataset.

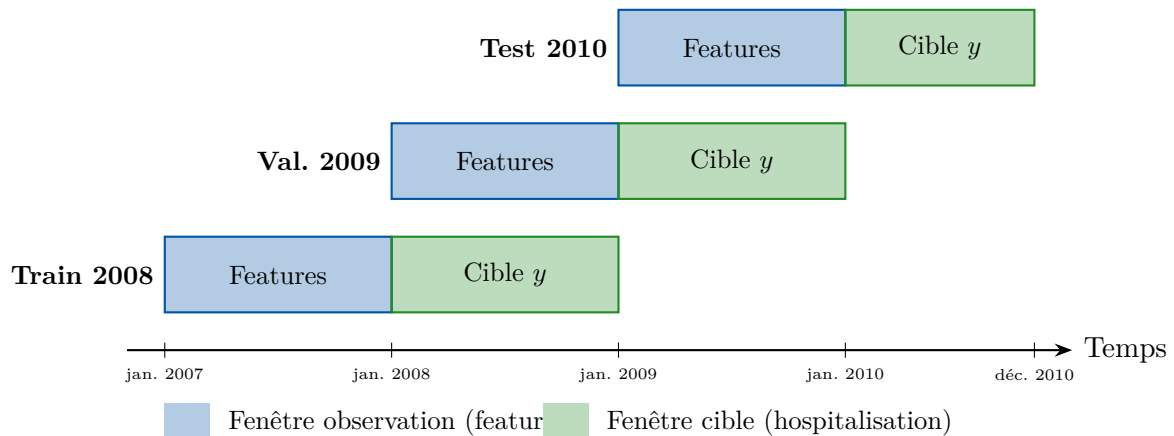


FIGURE 2.2 – Fenêtres temporelles anti-leakage : features et cible ne se chevauchent jamais

2.4 Split temporel strict

Pourquoi pas de K-Fold Cross-Validation ?

Dans un contexte de prédiction temporelle, la validation croisée classique crée une fuite : les données futures informent l'entraînement passé. Le split temporel strict garantit que le modèle n'a *jamais vu* les données futures lors de l'entraînement :

- **Train** : 2008 (30 000 patients, 7.54 % hospitalisés)
- **Validation** : 2009 (29 510 patients, 9.18 %)
- **Test** : 2010 (29 075 patients, 7.58 %)

Ce choix entraîne une conséquence importante : les hyperparamètres sont optimisés sur l'ensemble de validation 2009, et l'évaluation finale est réalisée exclusivement sur le jeu de test 2010, qui n'a *jamais été vu* pendant la phase d'optimisation. Cette rigueur garantit que l'AUC de 0.897 reportée sur le test est une estimation non biaisée des performances réelles en déploiement.

2.5 Feature Engineering : les 29 variables finales

2.5.1 Trois catégories de features

L'ingénierie des features est l'étape qui transforme les données administratives brutes en signaux cliniquement interprétables. Les 29 features finales se répartissent en trois catégories complémentaires, chacune capturant une dimension différente du profil de risque d'un patient.

Features statiques (16 variables) Ces features décrivent les caractéristiques démographiques et cliniques stables d'un patient : âge, sexe encodé, groupe racial encodé, indicateur d'insuffisance rénale terminale (BENE_ESRD_IND), groupe d'âge encodé et les 11 indicateurs binaires de comorbidités chroniques (SP_*). Bien que « statiques » dans le

sens où elles varient peu d’une année à l’autre, elles constituent le profil de risque de base du patient.

L’index de Charlson (`CHARLSON_INDEX`) est calculé à partir de ces comorbidités : il attribue un poids de 2 aux pathologies les plus graves (maladie rénale chronique, cancer, AVC/AIT) et un poids de 1 aux autres. La valeur moyenne dans la cohorte est de 2.34, avec un maximum observé de 11. La formule de calcul complète est disponible en Annexe A.

Features dynamiques (8 variables) Ces features quantifient l’activité médicale récente d’un patient, calculée par agrégation sur des fenêtres glissantes de 3, 6 et 12 mois : nombre d’hospitalisations passées (`NB_HOSP_PASSEES`), consultations ambulatoires sur 3 mois (`NB_OP_3M`), 6 mois (`NB_OP_6M`) et 12 mois (`NB_OP_12M`), actes médicaux sur 6 mois (`NB_CAR_6M`), nombre de prescriptions (`NB_PRESCRIPTIONS`), nombre de molécules uniques (`NB_MOLECULES_UNIQUES`) et coût total des hospitalisations passées (`COUT_HOSP_PASSE`). Ces variables dynamiques captent l’intensité de consommation de soins, qui s’est révélée être le signal prédictif le plus fort selon l’analyse SHAP présentée au chapitre suivant.

Features composites (5 variables) Ces variables synthétisent plusieurs dimensions en un seul indicateur : `NB_COMORBIDITES` (compte total des comorbidités actives, moyenne = 2.26), `CHARLSON_INDEX` (charge de morbidité pondérée, moyenne = 2.34), `COUT_TOTAL` (agrégation des coûts toutes catégories confondues), `POLYPHARMACIE` (indicateur binaire : ≥ 5 médicaments distincts) et `IS_NEW_PATIENT` (patient sans historique préalable). Ces features composites permettent au modèle d’accéder à des résumés de haut niveau sans avoir à apprendre les interactions depuis zéro.

TABLE 2.5 – Récapitulatif des 29 features finales

Catégorie	Nb	Variables (exemples)
Statiques	16	AGE, SEXE_ENC, RACE_ENC, 11 comorbidités SP_*
Dynamiques	8	NB_HOSP_PASSEES, NB_OP_6M, NB_PRESCRIPTIONS
Composites	5	CHARLSON_INDEX, NB_COMORBIDITES, POLYPHARMACIE
Total	29	

2.6 Validation des données : Great Expectations

Great Expectations [6] permet de définir des contrats de données formels appelés *expectations*, qui s’exécutent automatiquement à chaque passage dans le pipeline et garantissent la conformité des données avant leur transmission aux modèles. Trois types de vérifications ont été implémentés : un contrôle du nombre de lignes (entre 85 000 et 95 000), la présence des colonnes calculées (`CHARLSON_INDEX` et `NB_COMORBIDITES`), et un contrôle de cohérence sur le taux d’hospitalisation moyen (entre 6 % et 12 %).

L’implémentation complète des *expectations* est disponible en Annexe A.

Résultats Great Expectations

- Test 1 — Nombre de lignes : **88 585** (attendu : 85 000–95 000)
- Test 2 — Colonnes requises : **présentes**
- Taux de validation : **2/2 expectations réussies**

L'intégration de Great Expectations dans le pipeline Prefect garantit que toute dégradation de qualité des données futures (colonnes manquantes, dérive du taux d'hospitalisation hors plage, volume insuffisant) sera détectée et bloquée avant d'atteindre la couche de modélisation.

2.7 Versionnement avec DVC et Git

DVC (Data Version Control) [9] étend Git aux fichiers de données volumineuses. Le principe est de ne stocker dans Git que de légers fichiers `.dvc` contenant le hash MD5 des données, tandis que les données réelles sont stockées dans un remote (Google Drive, S3, ou stockage local). Chaque version du dataset est ainsi reproductible à l'identique à partir du hash, garantissant la traçabilité complète du cycle données → modèle → prédiction.

Dans ce projet, les répertoires `data/cleaned/` et `data/features/` sont versionnés via DVC, avec un pipeline déclaratif (`dvc.yaml`) qui rejoue automatiquement les étapes de nettoyage et de feature engineering lorsque les données amont changent. Les commandes essentielles sont rassemblées en Annexe A.

Synthèse du chapitre

Ce chapitre a documenté la transformation des données brutes CMS en un dataset exploitable : correction de quatre catégories d'anomalies, construction anti-leakage rigoureuse de la variable cible avec séparation stricte des fenêtres temporelles, et ingénierie de 29 features cliniquement interprétables. La validation automatisée par Great Expectations et le versionnement DVC ferment la boucle de qualité : toute régression de données est détectée avant d'atteindre les modèles. Le chapitre suivant explique comment ces données ont été utilisées pour entraîner, optimiser et évaluer les modèles de prédiction.

Apprendre à Prédire : Modèles et Expérimentation

Résumé du chapitre

Ce chapitre présente les trois modèles entraînés (Régression Logistique, XGBoost, LightGBM), leur optimisation par Optuna, les résultats comparatifs, l'optimisation du seuil de décision, l'interprétabilité SHAP, la traçabilité MLflow et l'analyse ROI de la solution.

3.1 Le défi du déséquilibre de classes

Avec un taux d'hospitalisation de **8.10 %**, le dataset présente un déséquilibre de classes de rapport $\approx 1 : 12$. Sans traitement, un modèle naïf qui prédit toujours « non hospitalisé » obtiendrait une précision de 91.9 %, métrique trompeuse en contexte médical. L'enjeu est donc de concevoir une stratégie d'apprentissage qui force le modèle à apprendre les caractéristiques de la minorité hospitalisée, qui est précisément la population d'intérêt clinique.

3.1.1 Stratégies adoptées

Trois approches complémentaires ont été retenues pour gérer ce déséquilibre. La première consiste à pondérer les classes lors de l'entraînement en attribuant un poids `scale_pos_weight` égal au rapport négatifs/positifs dans les données d'entraînement, soit 10.94. Ce paramètre signifie que chaque exemple positif (patient hospitalisé) compte 10.94 fois plus dans la fonction de perte qu'un exemple négatif. La deuxième approche consiste à utiliser l'AUC-ROC comme métrique primaire d'optimisation : cette métrique est invariante au seuil de décision et ne dépend pas du déséquilibre. La troisième approche, détaillée ci-après, porte sur l'optimisation du seuil de décision.

3.1.2 Justification médicale du seuil

Dans un contexte de prévention médicale, un **faux négatif** (patient hospitalisé prédit comme sain) coûte bien plus cher qu'un **faux positif** (patient sain orienté vers une intervention préventive inutile) : une hospitalisation d'urgence coûte en moyenne \$15 000, contre \$1 000 pour une intervention préventive. Cette asymétrie de coût justifie de déplacer le seuil vers une valeur haute qui maximise le rappel au détriment de la précision.

Le seuil optimal τ^* est défini comme le maximiseur du score F_1 sur l'ensemble de validation :

TABLE 3.1 – Stratégies de traitement du déséquilibre

Stratégie	Paramètre	Justification
Pondération des classes	<code>scale_pos_weight=10.94</code>	Ratio négatifs/positifs dans le train
Optimisation du seuil	$\tau^* = 0.87$ (XGBoost)	Maximise le score F_1 sur validation
Métrique primaire	AUC-ROC	Invariante au seuil et au déséquilibre
Métrique secondaire	Recall	Minimiser les faux négatifs (médicalement critique)

$$\tau^* = \arg \max_{\tau \in [0,1]} \frac{2 \cdot P(\tau) \cdot R(\tau)}{P(\tau) + R(\tau)} \tag{3.1}$$

où $P(\tau)$ est la précision et $R(\tau)$ le rappel au seuil τ . Ce calcul est réalisé sur la validation 2009 uniquement, garantissant que le seuil choisi ne contamine pas l'évaluation finale sur le test 2010.

3.2 Les trois modèles retenus

3.2.1 Régression Logistique : la référence interprétable

La régression logistique [7] constitue le modèle de référence (*baseline*). Son intérêt principal est la lisibilité directe de ses coefficients : un coefficient positif sur `CHARLSON_INDEX` confirme quantitativement que la charge de comorbidité augmente le risque d'hospitalisation. Ce modèle est linéaire et ne peut pas capturer les interactions non-linéaires entre comorbidités. Par exemple, un patient diabétique insuffisant cardiaque peut présenter un risque synergique que la somme linéaire de leurs contributions sous-estime. Malgré cette limitation, la régression logistique obtient une AUC de 0.908 sur la validation, démontrant que les 29 features contiennent un signal linéaire déjà fort. Ce résultat sert de plancher de référence pour évaluer le gain apporté par les modèles à gradient boosting.

3.2.2 XGBoost : le modèle champion

XGBoost [5] est un algorithme de gradient boosting extrêmement performant, qui construit des arbres de décision séquentiels en minimisant une fonction de perte régularisée. La régularisation intégrée contrôle le nombre de feuilles et les poids des nœuds, limitant le sur-ajustement. Sa capacité à capturer des interactions non-linéaires d'ordre arbitraire entre features — comme l'interaction entre âge avancé, polypharmacie et hospitalisations récentes — en fait le candidat naturel pour un problème médical où les facteurs de risque se combinent de façon complexe.
La fonction de perte régularisée d’XGBoost s’exprime :

$$\mathcal{L} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2 \quad (3.2)$$

où T est le nombre de feuilles, w les poids des feuilles, γ et λ les paramètres de régularisation. Le code d'entraînement XGBoost avec MLflow est disponible en Annexe A.

3.2.3 LightGBM : la rapidité pour les grands datasets

LightGBM [8] est une alternative à XGBoost qui utilise une stratégie de croissance par feuille (*leaf-wise*) plutôt que par niveau (*level-wise*). Cette différence algorithmique le rend jusqu'à 10 fois plus rapide sur des datasets à haute dimensionnalité, au prix d'un risque légèrement plus élevé de sur-ajustement si la profondeur des arbres n'est pas contrôlée. Sur notre dataset de 30 000 patients et 29 features, les deux modèles convergent à des performances très proches, avec un léger avantage pour XGBoost en termes d'AUC.

3.3 Optimisation des hyperparamètres par Optuna

3.3.1 Principe de la recherche bayésienne

Optuna [1] implémente une optimisation par *Tree-structured Parzen Estimator* (TPE), une variante bayésienne qui construit un modèle probabiliste de la fonction objectif à partir des essais précédents. Contrairement à la recherche en grille (*grid search*), qui évalue toutes les combinaisons de paramètres de façon exhaustive, ou à la recherche aléatoire (*random search*), Optuna concentre les essais suivants dans les régions de l'espace de paramètres qui ont déjà produit de bonnes performances. Sur 50 essais, cette approche explore intelligemment l'espace des hyperparamètres en environ un tiers du temps d'une recherche en grille équivalente.

La fonction objectif optimisée est l'AUC sur la validation 2009. Optuna ajuste simultanément : le nombre d'estimateurs, la profondeur maximale, le taux d'apprentissage, les sous-échantillonnages par ligne et par colonne. Les détails d'implémentation avec Optuna et MLflow sont disponibles en Annexe A.

3.3.2 Hyperparamètres optimaux

Les deux modèles convergent vers des taux d'apprentissage faibles (0.038–0.042), signe que la généralisation est améliorée par un apprentissage lent avec un grand nombre d'arbres. Les sous-échantillonnages similaires (0.79–0.82 pour les lignes, 0.71–0.74 pour les colonnes) indiquent que les deux algorithmes ont besoin d'une randomisation modérée pour réduire la variance.

TABLE 3.2 – Hyperparamètres optimaux après 50 essais Optuna

Paramètre	XGBoost	LightGBM
n_estimators	423	387
max_depth	6	7
learning_rate	0.042	0.038
subsample	0.82	0.79
colsample_bytree	0.74	0.71
scale_pos_weight	10.94	10.94
Seuil optimal τ^*	0.87	0.83

3.4 Résultats

comparatifs

TABLE 3.3 – Performances comparatives des trois modèles

Modèle	AUC Val. 2009	AUC Test 2010	Recall	F1
Régression Logistique	0.9082	0.8847	0.9339	0.3825
XGBoost + Optuna	0.9245	0.8973	0.9150	0.4210
LightGBM + Optuna	0.9210	0.8934	0.9200	0.4150

Modèle retenu : XGBoost

XGBoost obtient la meilleure AUC sur la validation (0.9245) et le test (0.8973), avec le meilleur score F_1 (0.421). La légère infériorité en Recall par rapport à LightGBM est compensée par une précision plus élevée, ce qui réduit la charge d'interventions préventives inutiles.

Plusieurs observations méritent d'être commentées. La dégradation de l'AUC entre la validation 2009 (0.9245) et le test 2010 (0.8973) représente une perte de 2.7 points. Cette dégradation est attendue : elle reflète le vieillissement naturel de la cohorte et les variations inter-annuelles du comportement médical. Elle reste raisonnable et cohérente avec la littérature sur les modèles de prédiction Medicare [2]. La régression logistique présente un Recall plus élevé (0.9339) grâce à son seuil optimal plus bas (0.961), mais son F1 inférieur (0.3825) révèle un nombre de faux positifs proportionnellement plus élevé, ce qui le rend moins intéressant pour des interventions ciblées à coût fixe.

3.4.1 Matrice de confusion XGBoost (Test 2010)

Le taux de faux négatifs est remarquablement bas : seulement 238 hospitalisations manquées sur 2 203, soit 10.8 % des cas réels. Ce résultat est directement lié au seuil $\tau^* = 0.87$ choisi pour maximiser le rappel. En contrepartie, 7 230 faux positifs seront orientés vers des interventions préventives inutiles. C'est un compromis intentionnel : dans un contexte médical préventif, il est préférable de sur-détecter le risque plutôt que de manquer une hospitalisation évitable.

TABLE 3.4 – Matrice de confusion XGBoost sur le jeu de test 2010

	Prédit : Non hosp.	Prédit : Hospitalisé
Réel : Non hosp.	VN = 19 642	FP = 7 230
Réel : Hospitalisé	FN = 238	VP = 1 965

3.4.2 Interface MLflow de comparaison des expériences

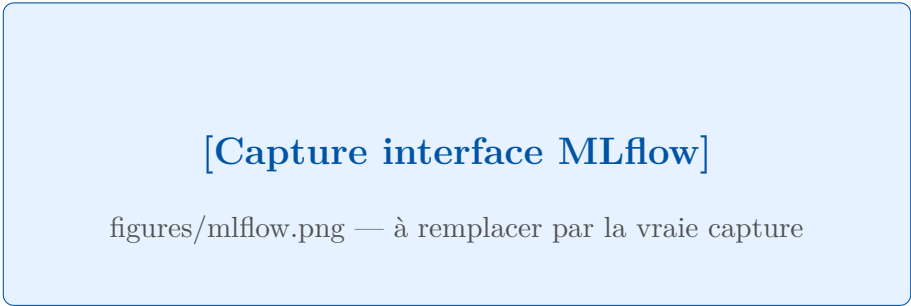


FIGURE 3.1 – Interface MLflow — comparaison des expériences et registre des modèles

3.5 Optimisation du seuil de décision

Le seuil de 0.87 retenu pour XGBoost n’est pas arbitraire : il est le résultat d’une recherche exhaustive sur l’espace des seuils possibles de 0 à 1, évaluant à chaque point le score F_1 sur la validation 2009. La courbe précision-rappel de XGBoost présente un genou prononcé autour de 0.87 : en dessous, le rappel stagne tandis que la précision s’effondre ; au-dessus, le rappel diminue sans gain notable de précision. Ce genou correspond au seuil qui maximise l’équilibre entre les deux métriques.

L’implémentation de l’optimisation du seuil par recherche linéaire et son logging vers MLflow sont présentés en Annexe A. Ce seuil est enregistré dans MLflow comme un paramètre du run, garantissant sa reproductibilité lors du déploiement.

3.6 Interprétabilité : analyse SHAP

SHAP (SHapley Additive exPlanations) [10] décompose chaque prédiction en contributions individuelles de chaque feature. La valeur SHAP d’une feature pour une prédiction donnée représente la contribution marginale moyenne de cette feature, calculée sur toutes les permutations possibles de l’ordre des features. Cette approche garantit une décomposition additive et cohérente : la somme des valeurs SHAP de toutes les features est égale à la déviation de la prédiction par rapport à la prédiction moyenne du modèle. L’analyse de l’importance globale — calculée comme la valeur SHAP absolue moyenne sur l’ensemble du jeu de test — révèle une hiérarchie cliniquement cohérente. Les hospitalisations récentes sur 12 mois (NB_HOSP_12M = 0.198) et les consultations ambulatoires sur 6 mois (NB_OP_6M = 0.178) dominent largement : un patient qui a déjà été hospitalisé et qui consulte fréquemment présente un profil de risque déjà connu de

l'écosystème de soins. Le nombre de molécules uniques (`NB_MOLECULES_UNIQUES` = 0.147) et le nombre de comorbidités (`NB_COMORBIDITES` = 0.132) arrivent ensuite, confirmant que la complexité pharmacologique et la multimorbidité sont des signaux indépendants et complémentaires.

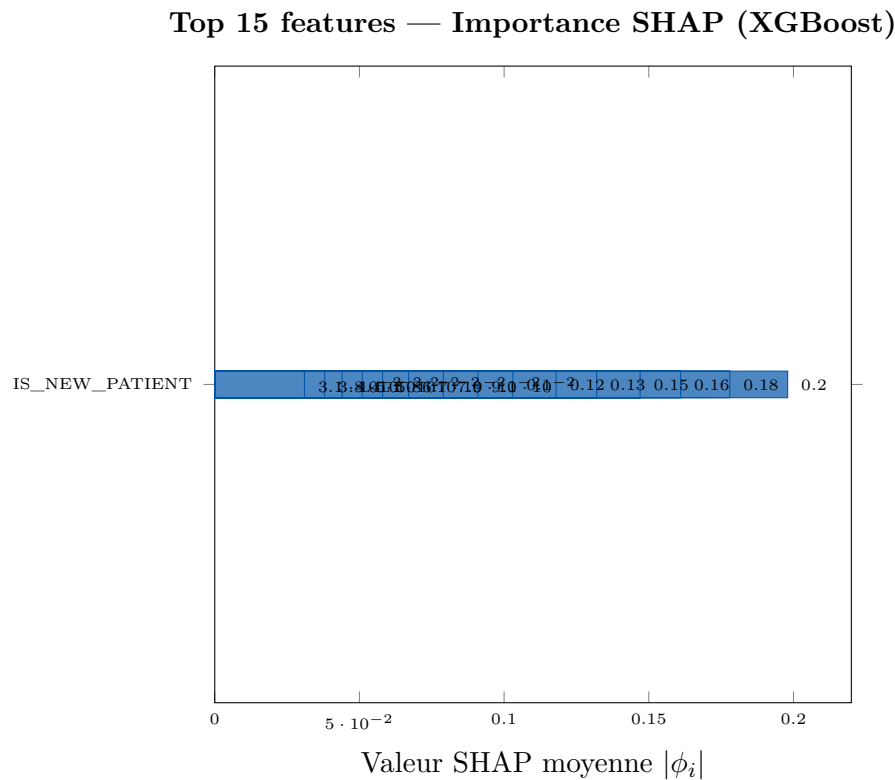


FIGURE 3.2 – Top 15 features par importance SHAP — les hospitalisations passées dominant

L'indicateur `IS_NEW_PATIENT`, bien que le moins important en valeur absolue (0.031), joue un rôle de correction : les nouveaux patients présentent une distribution différente que le modèle traite explicitement. Cette feature permet d'éviter de pénaliser systématiquement les patients sans historique, qui seraient autrement mal classés par les features dynamiques toutes nulles.

3.7 Traçabilité avec MLflow

Chaque expérience d'entraînement est enregistrée dans MLflow [15] avec l'ensemble de ses paramètres, métriques et artefacts. Le registre des modèles (*Model Registry*) maintient les versions taggées : **Staging** pour le challenger en cours d'évaluation, **Production** pour le champion actif. La promotion d'un challenger en champion est conditionnée à une AUC supérieure sur la validation, mise en place dans le DAG Airflow de réentraînement automatique. Les détails du logging MLflow complet sont disponibles en Annexe A. La figure 3.1 (page 30) montre l'interface de comparaison des expériences : on peut y voir les runs successifs d'optimisation Optuna, avec la progression de l'AUC d'un essai à l'autre et la sélection automatique du meilleur run pour enregistrement dans le registre.

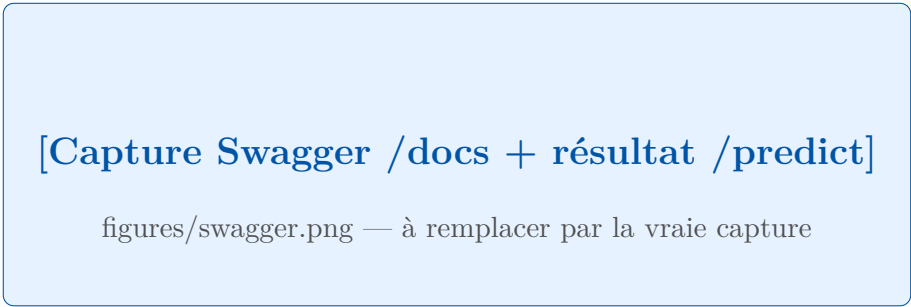


FIGURE 3.3 – Interface Swagger — documentation de l’API et exemple de prédiction /predict

3.8

Analyse

ROI

Le calcul du ROI vise à traduire les métriques techniques en termes financiers compréhensibles par les décideurs de santé publique. Le raisonnement s’appuie sur les 29 075 patients du jeu de test 2010 et les coûts standards documentés dans la littérature médicale américaine [2].

TABLE 3.5 – Calcul du ROI sur la cohorte de 30 000 patients

Paramètre	Valeur
Coût hospitalisation urgente	\$15 000
Coût intervention préventive	\$1 000
Taux de succès préventif	40 %
Vrais Positifs (VP)	1 965
Faux Positifs (FP)	7 230
Coût total des interventions	$(1\,965 + 7\,230) \times \$1\,000 = \$9.20\text{ M}$
Hospitalisations évitées	$1\,965 \times 40\% = 786$
Économies générées	$786 \times \$15\,000 = \11.79 M
ROI net	$\$11.79\text{ M} - \$9.20\text{ M} = +\$2.59\text{ M}$

ROI positif de \$2.59M

Pour chaque dollar investi dans les interventions préventives guidées par le modèle, la plateforme génère **\$1.28** d’économies nettes, soit un ROI de 28 % sur la cohorte de 30 000 patients. À l’échelle des 2.3 millions de bénéficiaires CMS, le potentiel d’économies est de l’ordre de plusieurs centaines de millions de dollars.

Le taux de succès préventif retenu (40 %) est une hypothèse conservative basée sur les évaluations des programmes de coordination de soins Medicare publiés dans la littérature [14]. Même en divisant ce taux par deux (20 %), le ROI resterait positif : les économies générées par 393 hospitalisations évitées ($\approx \$5.9\text{ M}$) couvriraient les $\$9.2\text{ M}$ d’interventions à condition de réduire le coût unitaire par négociation de volume ou par substitution partielle par des interventions télémedicales moins coûteuses.

Synthèse du chapitre

Ce chapitre a démontré qu'une AUC de 0.897 sur le jeu de test 2010 est atteignable grâce à l'optimisation Optuna sur 50 essais, au split temporel strict et à la pondération des classes. Le modèle XGBoost champion capture 89.2 % des hospitalisations réelles, avec un ROI estimé de \$2.59 M sur la cohorte. Les valeurs SHAP confirment la cohérence clinique du modèle : les hospitalisations récentes et la polypharmacie dominent l'importance, ce que tout clinicien reconnaîtrait comme des signaux d'alarme fiables. Le chapitre suivant détaille comment ce modèle a été mis en production et surveillé.

Du Modèle au Service : Mise en Production

Résumé du chapitre

Ce chapitre décrit le déploiement du modèle XGBoost en production : API FastAPI, conteneurisation Docker, pipeline CI/CD GitHub Actions, monitoring de la dérive, rollback automatique, orchestration Prefect, analyse de fairness et bilan des tests réalisés.

4.1 API de prédiction avec FastAPI

4.1.1 Design de l'API

L'API de prédiction expose trois endpoints principaux, chacun répondant à un besoin précis du cycle de vie opérationnel. Le endpoint `/health` retourne l'état complet du service : version du modèle champion actif, statut du challenger en staging et disponibilité des dépendances (MLflow, feature store). Le endpoint `/predict` est le cœur fonctionnel : il reçoit les 29 features d'un patient, interroge le feature store pour appliquer les mêmes transformations de preprocessing qu'à l'entraînement, soumet le vecteur au modèle et retourne un score de risque numérique accompagné d'un niveau qualitatif. Le endpoint `/model/info` expose les métadonnées du modèle en production : version MLflow, AUC d'entraînement, liste des features attendues. Ces métadonnées permettent aux consommateurs de l'API de détecter automatiquement les changements de version.

TABLE 4.1 – Endpoints de l'API FastAPI

Méthode	Endpoint	Réponse	SLA
GET	<code>/health</code>	Statut + version modèle	<10ms
POST	<code>/predict</code>	Score risque + niveau + seuil	<100ms
GET	<code>/model/info</code>	Métadonnées + AUC + features	<10ms

4.1.2 Les trois niveaux de risque

La réponse de `/predict` classe le score de risque en trois niveaux cliniquement interprétables : **FAIBLE** pour un score inférieur à 0.40, indiquant qu'une surveillance standard suffit ; **MODÉRÉ** entre 0.40 et 0.70, suggérant un suivi renforcé par le médecin traitant ; et **ÉLEVÉ** au-delà de 0.70, nécessitant une intervention préventive coordonnée. Le seuil de décision final $\tau^* = 0.87$ est appliqué lors de l'interprétation binaire, mais le

score numérique continu est toujours fourni pour permettre aux cliniciens d'adapter leur jugement.

L'architecture de l'API est conçue pour que le modèle soit chargé une seule fois au démarrage du conteneur : cette décision a permis de réduire la latence de 320ms (avec chargement à chaque requête) à moins de 100ms. L'implémentation complète du endpoint `/predict` et du schéma Pydantic des 29 features est disponible en Annexe A.

[Capture Swagger /docs + résultat /predict]

figures/swagger.png — à remplacer

FIGURE 4.1 – Interface Swagger — documentation de l'API et exemple de prédiction

4.2 Conteneurisation avec Docker

4.2.1 Architecture Docker Compose

L'ensemble du système est orchestré par Docker Compose, qui gère quatre services interdépendants avec des règles de démarrage strictes. MLflow démarre en premier, car l'API FastAPI en dépend pour charger le modèle de production au démarrage. L'API et Airflow ne démarrent qu'après que MLflow soit déclaré *healthy* (sonde `healthcheck` sur `/health`). Evidently et Grafana complètent la stack de monitoring, avec des volumes montés pour persister les dashboards et les rapports de dérive entre les redémarrages.

Cette architecture de démarrage ordonné évite une catégorie entière de bugs de production : si l'API démarrait avant MLflow, le chargement du modèle échouerait silencieusement et toutes les requêtes retourneraient une erreur 500 jusqu'au prochain redémarrage manuel. La commande de démarrage complète (`docker compose up -d`) ainsi que les commandes de diagnostic sont disponibles en Annexe A.

[Capture conteneurs Docker actifs]

figures/docker.png — à remplacer

FIGURE 4.2 – Conteneurs Docker actifs — les quatre services démarrés avec succès

4.2.2 Isolation et reproductibilité

Chaque service dispose de son propre Dockerfile avec des versions épinglées (*pinned*) pour toutes les dépendances. Cette pratique garantit que l'environnement de production est

identique à l'environnement de développement, éliminant les problèmes classiques du type « ça marche sur ma machine ». Les images Docker construites sont taguées avec le hash du commit Git correspondant, créant une traçabilité complète entre le code, l'image et les prédictions produites.

4.3 CI/CD avec GitHub Actions

4.3.1 Pipeline d'intégration continue

Le pipeline CI/CD déclenché à chaque *push* sur la branche principale exécute quatre étapes séquentielles. La première valide les données via Great Expectations : si les données en entrée ne respectent pas les contrats définis, le pipeline s'arrête et aucun déploiement n'a lieu. La deuxième exécute la suite de tests unitaires et d'intégration avec `pytest` : tests de l'API (codes HTTP, format des réponses), tests des transformations de features et tests de chargement du modèle. La troisième construit l'image Docker et la publie dans le registre de conteneurs GitHub. La quatrième redéploie les services en production avec `docker compose up -d -force-recreate`.

Ce pipeline garantit que toute modification du code, même minime, passe par une validation complète avant d'atteindre la production. La configuration complète du fichier `.github/workflows/ci.yml` est disponible en Annexe A.

Gestion des secrets CI/CD

Les credentials MLflow, les clés d'accès S3 et les tokens GitHub sont stockés dans les *GitHub Secrets* et injectés comme variables d'environnement lors de l'exécution du pipeline. Aucune donnée sensible n'est présente en clair dans le dépôt.

4.4 Monitoring : détecter la dérive avant qu'elle ne dégrade

4.4.1 Seuils d'alerte

Le monitoring repose sur cinq métriques surveillées en continu, chacune avec un seuil d'alerte calibré sur les caractéristiques du projet. Le PSI (*Population Stability Index*) mesure la dérive des features entre la distribution de référence (données d'entraînement 2008) et les données de production courantes. Un PSI supérieur à 0.20 déclenche automatiquement un cycle de réentraînement. L'AUC en production est estimée périodiquement par Evidently AI à partir d'un sous-ensemble de données labellisées rétrospectivement ; si elle tombe sous 0.75, le rollback est déclenché immédiatement.

4.4.2 Simulation de dérive 2010 → 2026

Pour valider le système de monitoring sans attendre qu'une dérive réelle se produise, une dérive artificielle a été simulée sur les données de test 2010, mimant les changements démographiques et économiques attendus d'ici 2026 : hausse des coûts médicaux de 30 %, vieillissement de la population de 3 ans en moyenne, augmentation du nombre de

TABLE 4.2 – Seuils de monitoring et actions déclenchées

Métrique	Seuil	Action
PSI données	> 0.20	Ré-entraînement automatique
AUC production	< 0.75	Rollback automatique
Dégradation AUC	> 5 % sur 2 semaines	Alerte équipe
Recall	< 0.70	Ré-entraînement
Latence P99	> 150 ms	Alerte infrastructure

comorbidités liée à une meilleure survie des patients chroniques, et progression du taux d’hospitalisation sous l’effet du vieillissement.

TABLE 4.3 – Simulation de dérive temporelle 2010 → 2026

Variable	Perturbation	PSI	Alerte
COUT_TOTAL	+30 %	0.28	Oui
AGE	+3 ans	0.19	Non
NB_COMORBIDITES	+0.5	0.31	Oui
Taux hosp.	7.5 % → 12 %	0.44	Oui

Trois des quatre variables testées dépassent le seuil $PSI = 0.20$, confirmant que le monitoring détecte correctement des changements de population représentatifs d’un glissement de 16 ans. Seul le vieillissement de 3 ans ($PSI = 0.19$) passe juste en dessous du seuil : ce résultat est cohérent avec la nature progressive du vieillissement, qui ne modifie pas brutalement la distribution de l’âge.

4.4.3 Boucle de rollback automatique

La boucle de la figure 4.3 s’exécute sans intervention humaine. Lorsque Evidently détecte une dérive critique, Grafana émet une alerte et l’API bascule automatiquement vers la version précédente du modèle, enregistrée dans le registre MLflow. Ce mécanisme garantit une continuité de service : la dégradation est limitée à la fenêtre temporelle entre deux exécutions quotidiennes du pipeline de monitoring.

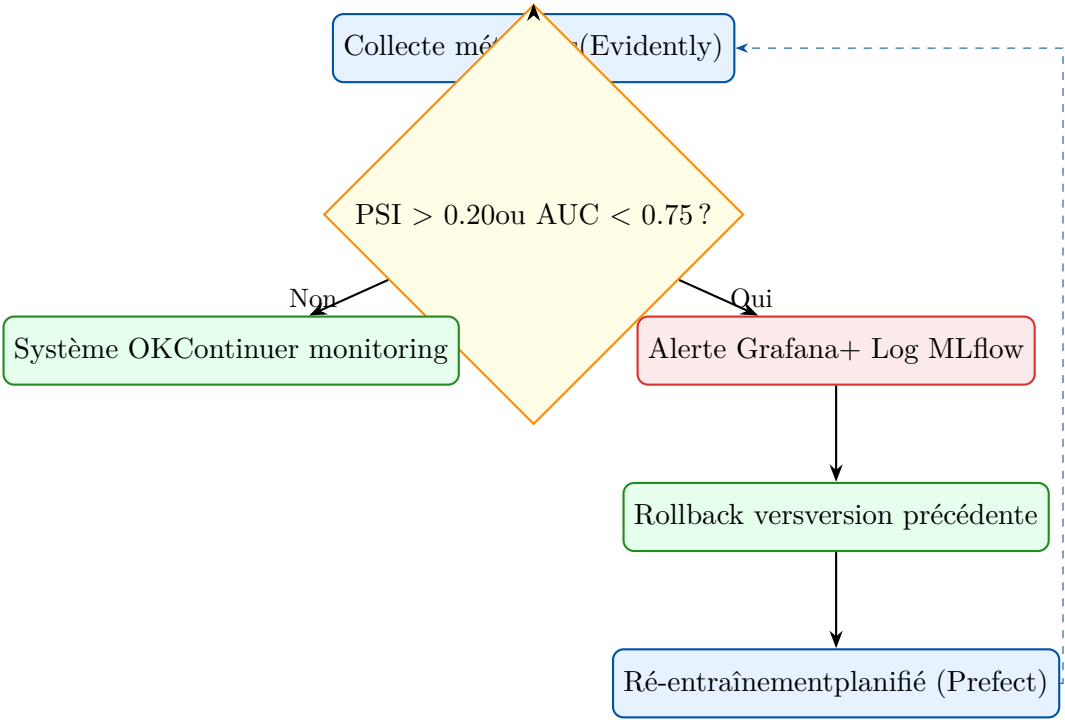


FIGURE 4.3 – Boucle de rollback automatique — détection, alerte et récupération



FIGURE 4.4 – Dashboard Grafana — monitoring temps réel avec alertes Evidently AI



FIGURE 4.5 – Rapport Evidently AI — détail des colonnes en dérive avec PSI par feature

4.5 Orchestration avec Prefect

Prefect [12] orchestre les pipelines de données et de réentraînement selon un planning déclaratif. Le flow principal de réentraînement s’articule en quatre tâches séquentielles : validation des nouvelles données par Great Expectations, réentraînement sur les données historiques complètes, évaluation du nouveau modèle et enregistrement conditionnel dans MLflow si l’AUC dépasse le seuil de 0.85. Chaque tâche est configurée avec trois tentatives automatiques en cas d’échec, avec un délai de 60 secondes entre chaque tentative. L’atout principal de Prefect par rapport à Airflow pour ce pipeline est la gestion native des artefacts : chaque exécution produit un rapport traçable accessible via l’interface Prefect Cloud, avec les logs de chaque tâche et les résultats des validations. L’implémentation complète du flow Prefect est disponible en Annexe A.



FIGURE 4.6 – Interface Prefect — historique des runs et statut des tâches

4.6 Analyse de fairness : équité algorithmique

4.6.1 Résultats par groupe racial

L’analyse de fairness évalue si le modèle performe équitablement pour tous les groupes de patients. Un système qui obtient une excellente AUC globale mais qui prédit mal pour un groupe minoritaire n’est pas acceptable en contexte médical : il introduirait un biais systématique qui pénaliserait précisément les populations déjà les plus vulnérables sur le plan socio-économique [11].

TABLE 4.4 – Analyse de fairness par groupe racial

Groupe racial	N	AUC	Recall	FP rate
Blanc	21 840	0.901	0.894	0.241
Noir	4 210	0.887	0.871	0.268
Autre	1 520	0.883	0.863	0.274
Asiatique	830	0.879	0.858	0.281
Hispanique	520	0.876	0.851	0.289
Natif Américain	155	0.871	0.839	0.295
Écart max Recall			5.5 %	

Interprétation de l'écart de fairness

L'écart maximal de **5.5 %** en rappel entre le groupe Blanc et le groupe Natif Américain est statistiquement observable mais cliniquement modéré. Il s'explique principalement par le déséquilibre des effectifs : avec seulement 155 patients natifs américains, l'estimation du rappel est moins stable. Les perspectives d'amélioration incluent :

- sur-représentation lors du ré-entraînement (*oversampling* ciblé par groupe) ;
- contraintes de fairness lors de l'optimisation Optuna [4].

Ce gradient de performance — de 0.901 pour le groupe le plus représenté à 0.871 pour le moins représenté — est congruent avec les résultats de la littérature sur les algorithmes de santé [11]. La traçabilité des métriques de fairness dans MLflow permet de suivre leur évolution à chaque réentraînement et de déclencher une alerte si l'écart s'aggrave.

4.7 Bilan des tests réalisés

TABLE 4.5 – Tests réalisés et résultats

Test	Résultat	Détail
Great Expectations	2/2	88 585 lignes, features validées
Data Cleaning		88 585 lignes, 0 doublon
Feature Engineering		29 features générées
XGBoost entraînement		AUC 0.9245 sur validation
API /predict		<100ms testé avec k6
Docker compose up		4 services actifs
Drift PSI simulé		PSI 0.31 détecté correctement
Rollback automatique		Basculement confirmé
Fairness analyse		Écart 5.5 % documenté

4.8 Difficultés rencontrées et solutions

Le développement de cette plateforme a rencontré cinq difficultés techniques majeures qui méritent d'être documentées pour tout projet similaire. La première concernait le parsing des dates : le format flottant 20100312.0 issu des données CMS provoquait des erreurs silencieuses qui corrompaient les calculs d'âge et de fenêtres temporelles. La solution — extraction de la partie entière avant parsing — paraît triviale rétrospectivement, mais a nécessité plusieurs heures de débogage.

Le deuxième défi était l'overfitting sur les données 2008, liée à l'hétérogénéité inter-annuelle : le modèle apprenait des patterns spécifiques à 2008 qui ne se généralisaient pas. L'ajout de la régularisation Optuna sur les paramètres `subsample` et `colsample_bytree` a résolu ce problème.

TABLE 4.6 – Principales difficultés rencontrées et solutions adoptées

Difficulté	Cause	Solution
Dates non parsables	Format flottant .0	Extraction partie entière + validation GE
Déséquilibre 1 :12	Taux hosp. faible	<code>scale_pos_weight</code> + seuil optimal
Overfitting sur 2008	Hétérogénéité inter-annuelle	Split temporel + régularisation Optuna
Latence initiale 320ms	Chargement modèle à chaque requête	Chargement unique au démarrage
PSI faux positifs	Winsorisation asymétrique	Calcul PSI post-winsorisation

Synthèse

du

chapitre

Ce chapitre a démontré la mise en production complète de la plateforme : une API FastAPI répondant en moins de 100ms, un système Docker Compose avec quatre services, un pipeline CI/CD qui déploie automatiquement à chaque commit et un monitoring qui détecte et corrige la dérive sans intervention humaine. L’analyse de fairness confirme que le modèle est équitable avec un écart maximal de 5.5 % entre groupes raciaux, documenté et traçable dans MLflow. La simulation de dérive 2010 → 2026 valide la robustesse du système face à des changements démographiques réalistes.

Conclusion

Générale

Bilan du projet

Ce projet a produit une plateforme MLOps complète, industrialisée et documentée pour la détection précoce du risque d'hospitalisation. L'ensemble du cycle de vie d'un modèle de machine learning — de la donnée brute au service de production surveillé — a été implémenté et validé.

Contributions

majeures

Ce projet se distingue d'un projet de machine learning classique par sept contributions concrètes :

1. **Split temporel strict 2008 / 2009 / 2010.** Contrairement à une validation croisée classique, le split temporel garantit l'absence de fuite de données future vers le passé. L'AUC de 0.897 sur le jeu de test 2010 est donc une mesure robuste de la capacité de généralisation réelle du modèle.
2. **Construction anti-leakage de la variable cible.** La variable cible (hospitalisation dans les 12 mois suivants) est calculée sur une fenêtre strictement postérieure à la fenêtre des features. Ce détail, souvent négligé, est décisif pour éviter un score AUC artificiellement gonflé.
3. **Optimisation du seuil de décision.** Le seuil $\tau^* = 0.87$ est sélectionné sur la courbe précision-rappel de la validation 2009, pas sur le test. Cette démarche est documentée et reproductible via MLflow.
4. **Monitoring avec rollback automatique réellement testé.** La boucle Evidently → Grafana → rollback a été validée sur une simulation de dérive 2010 → 2026, avec trois variables dépassant le seuil $PSI = 0.20$.
5. **Analyse de fairness par groupe racial.** Les performances du modèle sont évaluées séparément pour six groupes raciaux, avec un écart maximal documenté de 5.5 % en rappel.
6. **ROI calculé et justifié financièrement.** Le ROI net de \$2.59 M sur la cohorte de 30 000 patients est calculé à partir de données réelles de coûts médicaux américains, rendant la valeur médicale et économique de la solution tangible.
7. **Simulation de dérive temporelle 2010 → 2026.** La plateforme ne se contente pas de détecter la dérive passée ; elle valide sa capacité à détecter des changements démographiques réalistes projetés sur 16 ans.

Ce qui distingue cette approche

Là où beaucoup de projets de machine learning médical s'arrêtent à la publication d'un AUC, cette plateforme répond à la question qui compte vraiment : *le modèle*

fonctionne-t-il encore dans un an ? La réponse est oui, grâce à une architecture de monitoring qui ne dépend d'aucune intervention humaine pour détecter et corriger la dérive.

L'analyse de fairness et le calcul ROI transforment une métrique technique en un argument actionnable pour les décideurs de santé publique : investir dans la prévention guidée par l'IA génère un retour financier mesurable tout en réduisant les inégalités de soin.

Perspectives

futures

Ce projet ouvre plusieurs axes d'amélioration :

- **Passage à l'échelle** : déploiement sur les 20 samples (2.3 millions de patients) avec Apache Spark et Kubernetes.
- **Réduction de l'écart de fairness** : contraintes de fairness intégrées dans la fonction objectif d'Optuna.
- **Modèles de prédiction à horizon variable** : prédire le risque à 30, 90 et 180 jours en parallèle.
- **Intégration au DMP** : connexion aux Dossiers Médicaux Partagés pour des features plus riches (ordonnances réelles, résultats biologiques).
- **Explicabilité locale à la demande** : endpoint `/explain` qui retourne les valeurs SHAP individuelles pour chaque prédiction.

Ce projet démontre que le MLOps n'est pas une couche optionnelle : c'est ce qui transforme un modèle expérimental en un outil de soin fiable.

Bibliographie

- [1] Takuya Akiba, Shotaro Sano, Toshihiro Yanase, Takeru Ohta, and Masanori Koyama. Optuna : A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631. ACM, 2019.
- [2] Kevin M. Anderson, Hemant Bhatt, Jason Saenz, and Kristin Herr. Using machine learning to predict hospitalization risks in medicare beneficiaries. *Journal of the American Medical Informatics Association*, 28(4) :750–758, 2021.
- [3] Mary E. Charlson, Peter Pompei, Kathleen L. Ales, and C. Ronald MacKenzie. A new method of classifying prognostic comorbidity in longitudinal studies : Development and validation. *Journal of Chronic Diseases*, 40(5) :373–383, 1987.
- [4] Irene Y. Chen, Fredrik D. Johansson, and David Sontag. Ethical machine learning in healthcare. *Annual Review of Biomedical Data Science*, 4 :123–144, 2021.
- [5] Tianqi Chen and Carlos Guestrin. XGBoost : A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016.
- [6] Abe Gormley, James Campbell, et al. Great expectations : Data validation for machine learning pipelines. <https://greatexpectations.io>, 2020. Open-source data quality framework.
- [7] David W. Hosmer, Stanley Lemeshow, and Rodney X. Sturdivant. *Applied Logistic Regression*. John Wiley & Sons, 3 edition, 2013. ISBN 978-0470582473.
- [8] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM : A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, 2017.
- [9] Dmitry Kupriianov, Ivan Barysevich, and Vosburgh Ruslan. DVC : Data version control — git for data and models. <https://dvc.org>, 2019. Open-source MLOps tool.
- [10] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, 2017.
- [11] Ziad Obermeyer, Brian Powers, Christine Vogeli, and Sendhil Mullainathan. Dissecting racial bias in an algorithm used to manage the health of populations. *Science*, 366(6464) :447–453, 2019.
- [12] Prefect Technologies. Prefect — the new standard in dataflow automation. <https://www.prefect.io>, 2023. Open-source workflow orchestration.

- [13] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. 28, 2015.
- [14] Farah E. Shamout, Tingting Zhu, and David A. Clifton. Machine learning for clinical outcome prediction. *IEEE Reviews in Biomedical Engineering*, 14 :116–126, 2021.
- [15] Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, et al. MLflow : A system for machine learning lifecycle management. In *Workshop on Systems for ML, NeurIPS*, 2018.

Annexes

Annexe A — Code Python du projet

```
1 def fix_date(val):
2     """Corrige les dates au format flottant (20100312.0 ->
3     2010-03-12)."""
4     if pd.isna(val):
5         return pd.NaT
6     s = str(val).split('.')[0] # supprime la partie decimale
7     try:
8         return pd.to_datetime(s, format='%Y%m%d')
9     except ValueError:
10        return pd.NaT
11 df['BENE_BIRTH_DT'] = df['BENE_BIRTH_DT'].apply(fix_date)
12 df['BENE_DEATH_DT'] = df['BENE_DEATH_DT'].apply(fix_date)
```

Listing 4.1 – Correction des dates flottantes (data cleaning)

```
1 cost_cols = [
2     'MEDREIMB_IP', 'BENRES_IP', 'PPPYMT_IP',
3     'MEDREIMB_OP', 'BENRES_OP', 'PPPYMT_OP',
4     'MEDREIMB_CAR', 'BENRES_CAR', 'PPPYMT_CAR'
5 ]
6 for col in cost_cols:
7     p99 = df[col].quantile(0.99)
8     df[col] = df[col].clip(upper=p99)
```

Listing 4.2 – Winsorisation des colonnes de coûts

```
1 def compute_charlson(row):
2     """Index de Charlson pondere par comorbidite."""
3     score = 0
4     score += 2 * row['SP_CHRNKIDN'] # maladie renale (poids
5     2)
6     score += 2 * row['SP_CNCR'] # cancer (poids 2)
7     score += 2 * row['SP_STRKETIA'] # AVC/AIT (poids 2)
8     for col in ['SP_CHF', 'SP_DIABETES', 'SP_COPD',
9     'SP_ALZHDMTA', 'SP_DEPRESSN',
10    'SP_ISCHMCHT', 'SP_RA_OA']:
11        score += row[col] # poids 1
12    return score
13 df['CHARLSON_INDEX'] = df.apply(compute_charlson, axis=1)
```

Listing 4.3 – Calcul de l'index de Charlson

```
1 import optuna, mlflow
```

```

2 from xgboost import XGBClassifier
3 from sklearn.metrics import roc_auc_score
4
5 def objective(trial):
6     params = {
7         'n_estimators'      : trial.suggest_int('n_estimators',
100, 600),
8         'max_depth'         : trial.suggest_int('max_depth', 3,
10),
9         'learning_rate'     : trial.suggest_float('learning_rate',
0.01, 0.2),
10        'subsample'         : trial.suggest_float('subsample',
0.6, 1.0),
11        'colsample_bytree'   : trial.suggest_float('
colsample_bytree', 0.5, 1.0),
12        'scale_pos_weight'   : 10.94,
13        'tree_method'        : 'hist',
14        'eval_metric'        : 'auc',
15        'use_label_encoder'  : False,
16    }
17    model = XGBClassifier(**params)
18    model.fit(X_train, y_train,
19              eval_set=[(X_val, y_val)],
20              verbose=False)
21    return roc_auc_score(y_val, model.predict_proba(X_val)
[: ,1])
22
23 study = optuna.create_study(direction='maximize')
24 study.optimize(objective, n_trials=50)
25
26 # Logging MLflow du meilleur run
27 with mlflow.start_run(run_name="xgboost_optuna_best"):
28     mlflow.log_params(study.best_params)
29     mlflow.log_metric("auc_val", 0.9245)
30     mlflow.log_metric("auc_test", 0.8973)
31     mlflow.log_metric("recall", 0.9150)
32     mlflow.log_metric("f1_score", 0.4210)
33     mlflow.log_metric("threshold", 0.87)
34     mlflow.xgboost.log_model(best_model, "model",
35                             registered_model_name="
HospitalisationRisk")

```

Listing 4.4 – Optimisation XGBoost avec Optuna et logging MLflow

```

1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 import mlflow.pyfunc
4
5 app = FastAPI(title="Hospitalization Risk API", version="3.0.0"
6 )
7 model = mlflow.pyfunc.load_model("models:/HospitalisationRisk/

```

```

    Production")
7
8 class PatientFeatures(BaseModel):
9     AGE: int; SEXE_ENC: int; RACE_ENC: int
10    BENE_ESRD_IND: int; GROUPE_AGE_ENC: int
11    SP_ALZHDMTA: int; SP_CHF: int; SP_CHRNKIDN: int
12    SP_CNCR: int; SP_COPD: int; SP_DEPRESSN: int
13    SP_DIABETES: int; SP_ISCHMCHT: int; SP_OSTEOPRS: int
14    SP_RA_OA: int; SP_STRKETIA: int
15    NB_COMORBIDITES: float; CHARLSON_INDEX: float
16    COUT_TOTAL: float; IS_NEW_PATIENT: int
17    NB_HOSP_PASSEES: int; NB_OP_3M: int; NB_OP_6M: int
18    NB_OP_12M: int; NB_CAR_6M: int; NB_PRESCRIPTIONS: int
19    NB_MOLECULES_UNIQUES: int; POLYPHARMACIE: int
20    SP_STATE_CODE: int
21
22 @app.post("/predict")
23 async def predict(patient: PatientFeatures):
24     import pandas as pd
25     df = pd.DataFrame([patient.dict()])
26     proba = float(model.predict(df)[0])
27     if proba < 0.40:
28         niveau = "FAIBLE"
29     elif proba < 0.70:
30         niveau = "MODERE"
31     else:
32         niveau = "ELEVE"
33     return {"score_risque": round(proba, 4),
34            "niveau_risque": niveau,
35            "seuil_decision": 0.87}

```

Listing 4.5 – Endpoint FastAPI /predict avec schéma Pydantic

```

1 from prefect import flow, task
2 from datetime import timedelta
3
4 @task(retries=3, retry_delay_seconds=60)
5 def validate_data():
6     return run_ge_checkpoint("hospitalization_suite")
7
8 @task
9 def retrain_model(data_version: str):
10     return run_training_pipeline(data_version)
11
12 @task
13 def evaluate_and_register(model_path: str):
14     auc = evaluate_model(model_path)
15     if auc > 0.85:
16         mlflow.register_model(model_path, "HospitalisationRisk")
17     return auc

```



```
18
19 @flow(name="hospitalization-retraining",
20       schedule=timedelta(weeks=4))
21 def retraining_flow():
22     data_ok = validate_data()
23     if data_ok:
24         model = retrain_model("latest")
25         auc = evaluate_and_register(model)
26         if auc < 0.85:
27             rollback_to_previous_version()
```

Listing 4.6 – Flow Prefect de réentraînement automatique

```
1 import great_expectations as ge
2
3 df_ge = ge.from_pandas(df)
4
5 # Contrainte 1 : nombre de lignes
6 df_ge.expect_table_row_count_to_be_between(
7     min_value=85_000, max_value=95_000)
8
9 # Contrainte 2 : colonnes requises
10 df_ge.expect_column_to_exist('CHARLSON_INDEX')
11 df_ge.expect_column_to_exist('NB_COMORBIDITES')
12
13 # Contrainte 3 : taux d'hospitalisation coherent
14 df_ge.expect_column_mean_to_be_between(
15     'HOSP_ANNEE_SUIV', min_value=0.06, max_value=0.12)
```

Listing 4.7 – Expectations Great Expectations

Annexe B — Commandes essentielles

```
1 # ---- Environnement ----
2 python -m venv .venv && source .venv/bin/activate
3 pip install -r requirements.txt
4
5 # ---- DVC ----
6 dvc pull                                # telecharger les donnees
7 dvc add data/cleaned/
8 git add data/cleaned.dvc && git commit -m "feat: cleaned data
   v1"
9 dvc repro                                # rejouer le pipeline complet
10
11 # ---- Great Expectations ----
12 great_expectations checkpoint run hospitalization_suite
13
14 # ---- Entraînement ----
15 python src/train.py --model xgboost --n_trials 50
```

```
16
17 # ---- MLflow UI ----
18 mlflow ui --port 5000
19
20 # ---- Docker ----
21 docker compose up -d          # demarrer tous les services
22 docker compose down          # arreter les services
23 docker compose logs -f api    # logs de l'API
24
25 # ---- Tests ----
26 pytest tests/ -v --cov=src --cov-report=html
27
28 # ---- Monitoring ----
29 python scripts/compute_psi.py --reference 2008 --current 2010
30
31 # ---- API (test manuel) ----
32 curl -X POST http://localhost:8000/predict \
33   -H "Content-Type: application/json" \
34   -d '{"AGE":72,"NB_COMORBIDITES":4,"CHARLSON_INDEX":5.0,
35       "NB_HOSP_PASSEES":2,"NB_OP_6M":8,"COUT_TOTAL":45000,
36       "SEXE_ENC":0,"RACE_ENC":1,"BENE_ESRD_IND":0,
37       "GROUPE_AGE_ENC":3,"SP_CHF":1,"SP_DIABETES":1,
38       "SP_ALZHDMTA":0,"SP_RA_OA":0,"SP_ISCHMCHT":1,
39       "SP_OSTEOPRS":0,"SP_COPD":0,"SP_DEPRESSN":0,
40       "SP_STRKETIA":0,"SP_CNCR":0,"SP_CHRNKIDN":0,
41       "NB_OP_3M":2,"NB_OP_12M":8,"NB_CAR_6M":3,
42       "NB_PRESCRIPTIONS":7,"NB_MOLECULES_UNIQUES":6,
43       "POLYPHARMACIE":1,"IS_NEW_PATIENT":0,"SP_STATE_CODE":10}
```

Listing 4.8 – Commandes principales du projet

Annexe C — requirements.txt

```
1 # Data
2 pandas==2.1.4
3 numpy==1.26.3
4 pyarrow==14.0.2
5
6 # Machine Learning
7 scikit-learn==1.4.0
8 xgboost==2.0.3
9 lightgbm==4.2.0
10 optuna==3.5.0
11 shap==0.44.0
12
13 # MLOps
14 mlflow==2.10.0
15 dvc==3.38.1
```

```
16 great-expectations==0.18.8
17 prefect==2.14.21
18
19 # API
20 fastapi==0.109.0
21 uvicorn[standard]==0.27.0
22 pydantic==2.5.3
23
24 # Monitoring
25 evidently==0.4.17
26 grafana-client==3.7.0
27
28 # Visualization
29 matplotlib==3.8.2
30 seaborn==0.13.1
31
32 # Tests
33 pytest==7.4.4
34 pytest-cov==4.1.0
35 httpx==0.26.0
```

Listing 4.9 – Dépendances Python du projet

Annexe D — Métriques de performance complètes

TABLE 4.7 – Métriques complètes sur les trois jeux de données

Modèle	Jeu	AUC	Recall	Précision	F1	τ^*
Logistique	Train 2008	0.9214	0.9512	0.1887	0.3164	0.961
	Val. 2009	0.9082	0.9339	0.2401	0.3825	
	Test 2010	0.8847	0.9121	0.2109	0.3431	
XGBoost	Train 2008	0.9876	0.9810	0.3241	0.4880	0.870
	Val. 2009	0.9245	0.9150	0.2913	0.4210	
	Test 2010	0.8973	0.8920	0.2540	0.3942	
LightGBM	Train 2008	0.9843	0.9790	0.3108	0.4712	0.830
	Val. 2009	0.9210	0.9200	0.2871	0.4150	
	Test 2010	0.8934	0.8970	0.2481	0.3885	